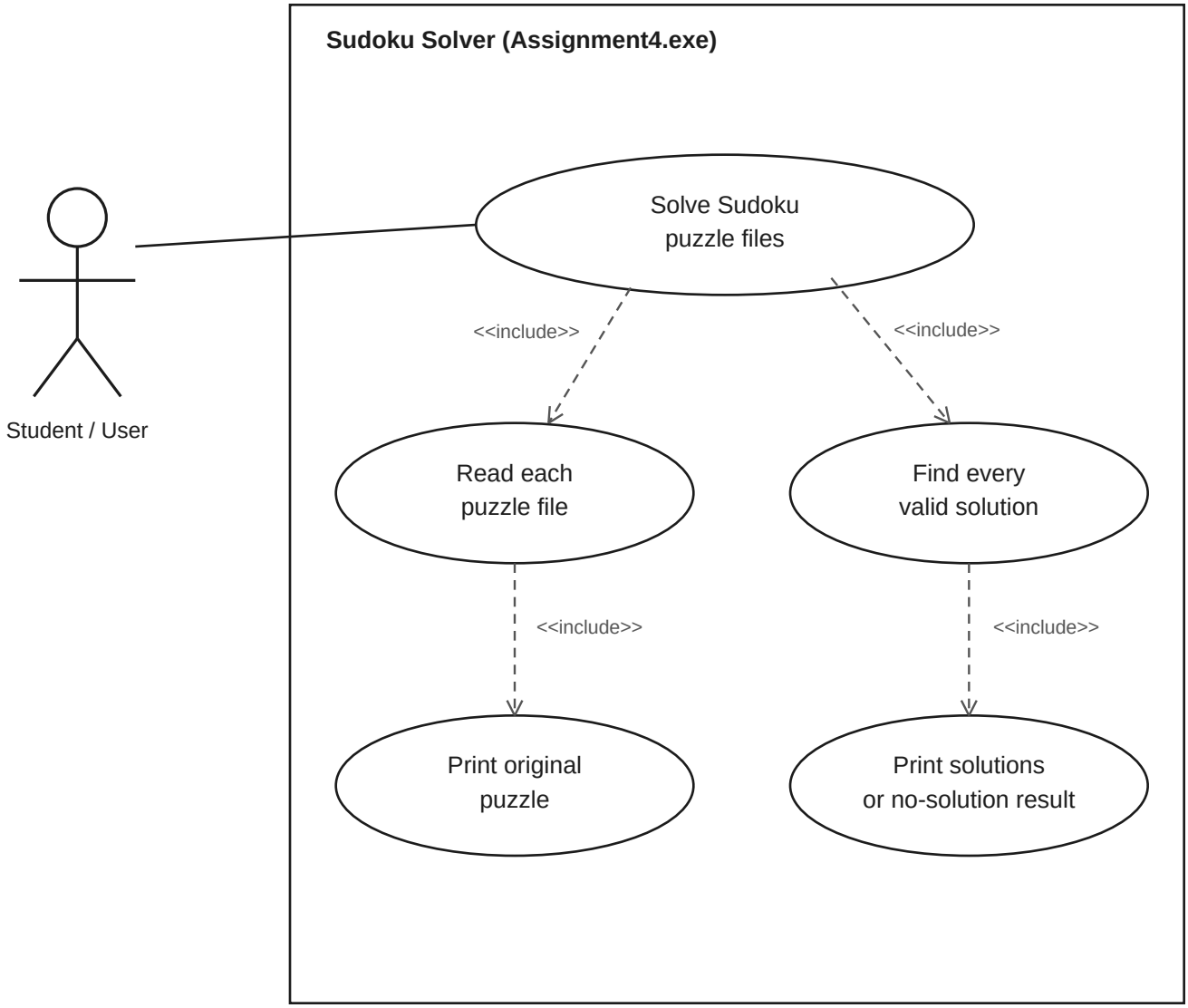
UML Use Case Diagram

Assignment 4: Recursive Sudoku Solver



Use case: Solve Sudoku puzzle files

The user runs the program. It reads five text files in order, prints each original puzzle, and uses recursive backtracking to find and display every valid solution.

Use Case Description

UC-01 | Solve Sudoku Puzzle Files

Primary actor	Student / User
Trigger	The user runs Assignment4.exe in the terminal.
Inputs	puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
Precondition	Puzzle files are expected in the program's working folder.

Brief description

The program works through five Sudoku puzzles. It reads each 9x9 grid, checks the filled numbers, and uses recursive backtracking to search for valid answers. It prints the original puzzle and each solution, or "No solution found" when a puzzle cannot be completed.

Main flow

1. The user runs the program from the terminal.
2. The program opens puzzle1.txt, then repeats the process for puzzles 2 through 5.
3. It reads the 9x9 grid, using underscores for empty cells.
4. It prints the filename and the original puzzle.
5. It checks that the numbers already filled in do not break Sudoku rules.
6. It chooses an empty cell, tries legal digits, and searches recursively.
7. If a choice fails, it backtracks and tries another digit.
8. Each complete solution is printed; the search continues for other solutions.
9. After finishing the current puzzle, the program moves to the next file.

Alternate outcomes

- Missing or invalid file: Prints "Unable to read puzzle file" and continues.
- Contradictory puzzle or no completion: Prints "No solution found".
- More than one solution: Prints each valid solution rather than stopping at the first.

Postcondition	Every provided puzzle file has been attempted and its outcome shown.

Diagram prepared with assistance from ChatGPT.